MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 9: IMAGE/VIDEO INPUT INTERFACING WITH MICROCONTROLLER

DATE OF EXPERIMENT: 10 DECEMBER 2025

DATE OF SUBMISSION: 17 DECEMBER 2025

SECTION 1

GROUP 13

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN

NO.	NAME	MATRIC
1	HARIZ IRFAN BIN MOHD ROZHAN	2318583
2	ABDULLAH HASAN BIN SIDEK	2318817
3	TAN YONG JIA	2319155
4	NUR QISTINA BINTI MOHD FAIZAL	2319512

ABSTRACT

This experiment focuses on video interfacing with a microcontroller by implementing a basic color detection system using an Arduino Uno, a Gravity HuskyLens AI Vision Camera, and an RGB LED. The system captures live image and video data through the HuskyLens, processes the visual information internally, and transmits detection results to the Arduino via serial communication. Based on the detected color ID, the Arduino controls an RGB LED to visually indicate the recognised color in real time. The experiment demonstrates fundamental concepts of video input interfacing, AI-based vision sensing, serial communication, and actuator control. System performance was evaluated based on detection reliability, response time, and consistency under controlled lighting conditions.

TABLE OF CONTENTS

INTRODUCTION

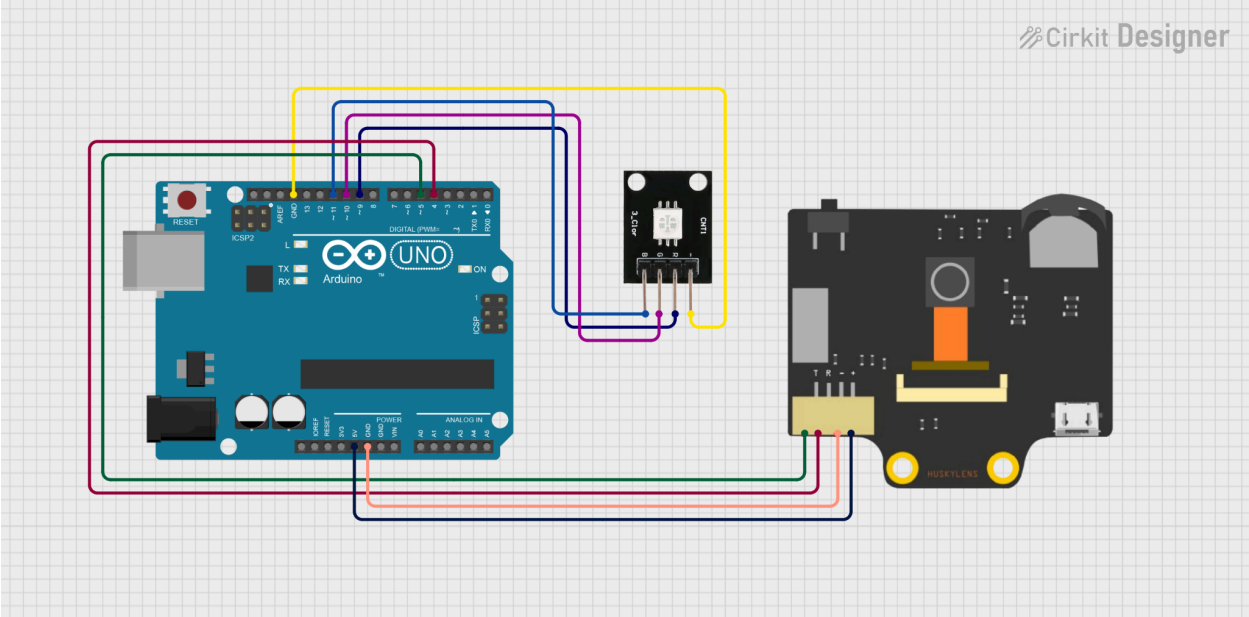
Video and image interfacing play a critical role in modern mechatronic and embedded systems, particularly in applications such as automation, robotics, quality inspection, and intelligent monitoring. Unlike conventional sensors that provide scalar data, vision-based sensors deliver rich spatial and contextual information, enabling more advanced decision-making.

In this Week 9 experiment, video interfacing is explored using the Gravity HuskyLens AI Vision Camera connected to an Arduino Uno. The HuskyLens is capable of real-time image processing and supports multiple vision algorithms, including color recognition. By interfacing this camera with a microcontroller, the system demonstrates how visual data can be acquired, interpreted, and utilized for control or monitoring purposes. This experiment provides hands-on exposure to serial communication, vision-based sensing, and practical AI-assisted perception in embedded systems.

MATERIALS AND EQUIPMENT

- Arduino Uno microcontroller board
- Gravity HuskyLens AI Vision Camera
- RGB LED (common cathode or common anode)
- Current-limiting resistors (typically 220 Ω)
- Jumper wires
- Breadboard (for wiring support)
- USB cable for Arduino programming and power
- Computer with Arduino IDE installed

EXPERIMENTAL SETUP



Circuit Design

METHODOLOGY

Hardware Configuration:

VCC of the HuskyLens AI Vision Camera was connected to the Arduino Uno 5V pin, while GND was connected to the Arduino ground to establish a common reference level. The HuskyLens TX pin was linked to Arduino digital pin D4, and the RX pin was connected to digital pin D5 using SoftwareSerial communication.

An all-in-one RGB LED was interfaced with the Arduino to provide visual feedback based on detected colors. The red, green, and blue pins of the RGB LED were each connected to separate Arduino digital output pins through individual 220 Ω current-limiting resistors. The common cathode of the RGB LED was connected to the common ground rail.

The Arduino Uno was powered via a USB connection from the computer, supplying a stable 5V input for both the HuskyLens camera and the RGB LED circuitry. To ensure reliable operation and stable signal reference levels, all components shared a common ground connection.

The RGB LED was used as an output indicator, where the Arduino activates the corresponding color channel when the HuskyLens detects a specific color (for example, red object detection triggers the red LED channel).

First, the HuskyLens camera was connected to the Arduino Uno using UART serial communication. SoftwareSerial was implemented to allow serial data exchange since the Arduino Uno has a single hardware serial port reserved for USB communication.

Next, an RGB LED was interfaced with the Arduino to serve as a visual output device. Each color channel (red, green, and blue) was connected to a separate digital output pin through a current-limiting resistor. This allowed the Arduino to display different colors by selectively activating the corresponding LED channels.

The HuskyLens was then configured in Color Recognition mode. Using the built-in learning function, the camera was trained to recognise three different colored objects. Each learned color was assigned a unique ID within the HuskyLens system.

An Arduino program was developed to continuously request detection data from the HuskyLens. When a color was detected, the Arduino read the returned color ID and activated the appropriate RGB LED channel. For example, detection of a red object caused the LED to illuminate red, while green and blue objects triggered their respective LED colors. Multiple trials were conducted to evaluate detection accuracy, LED response correctness, and system responsiveness.

Software Setup:

The Arduino IDE was used as the primary development environment. The following steps were performed:

- Installation of the Arduino Uno board package
- Installation of the DFRobot_HuskyLens library via the Arduino Library Manager
- Inclusion of the SoftwareSerial library for UART communication
- Configuration of multiple digital output pins for RGB LED control
- Configuration of baud rate at 9600 bps for stable data transmission

The program initializes communication with the HuskyLens and continuously sends detection requests. When a valid detection is received, the color ID is mapped to a specific RGB LED

output combination. The detected color ID and LED status are printed to the Serial Monitor for observation and debugging.

Physical Arrangement:

The Arduino Uno and HuskyLens camera were placed on a flat surface with the camera facing forward toward the target objects. Colored objects were positioned approximately 10-20 cm in front of the camera to ensure clear visibility and consistent detection.

The setup was arranged in a well-lit indoor environment to minimize the effect of shadows and inconsistent lighting. All wiring connections were kept short and secure to reduce signal noise and ensure reliable serial communication between the camera and the microcontroller.

Circuit Assembly:

The circuit assembly was completed using direct wiring between the Arduino Uno, HuskyLens camera, and the RGB LED:

HuskyLens connections:

- HuskyLens VCC connected to Arduino 5V
- HuskyLens GND connected to Arduino GND
- HuskyLens TX connected to Arduino digital pin D4 (SoftwareSerial RX)
- HuskyLens RX connected to Arduino digital pin D5 (SoftwareSerial TX)

RGB LED connections:

- Red pin connected to an Arduino digital output pin through a 220 Ω resistor
- Green pin connected to an Arduino digital output pin through a 220 Ω resistor
- Blue pin connected to an Arduino digital output pin through a 220 Ω resistor
- Common cathode connected to GND (or common anode connected to 5V, depending on LED type)

The Arduino Uno was powered via USB from the computer. Once powered, the system responded to detected colors by illuminating the RGB LED in the corresponding color, providing immediate visual feedback of the HuskyLens color recognition output.

Programming Logic

1. Library Initialization

The *SoftwareSerial* library was included to enable UART communication between the Arduino Uno and the HuskyLens AI Vision Camera using digital pins 4 and 5. The *DFRobot_HuskyLens* library was used to handle camera initialization, data requests, and result processing.

2. Pin Configuration

Three digital output pins were defined for the RGB LED:

- Pin 9 for the red LED channel
- Pin 10 for the green LED channel
- Pin 11 for the blue LED channel

These pins were configured as output pins to allow individual control of each LED color.

3. System Initialization (setup function)

Serial communication was initialized at 9600 baud for both the Serial Monitor and HuskyLens communication.

All RGB LED pins were set as OUTPUT and turned OFF during startup to prevent unintended illumination.

The Arduino continuously attempted to establish communication with the HuskyLens camera. If the camera was not detected, an error message was displayed on the Serial Monitor. Once communication was successfully established, a confirmation message indicating that the HuskyLens was ready was printed.

4. Video Data Request and Processing

Within the main loop, the Arduino continuously requested detection data from the HuskyLens. If the request failed, an error message was printed and the current loop iteration was terminated to avoid invalid data processing.

When detection data was available, the Arduino read the detected object's color ID along with its X and Y center coordinates. These values were displayed on the Serial Monitor for monitoring and debugging purposes.

5. RGB LED Control Logic

Before displaying any color, all RGB LED channels were turned OFF to ensure that only one color was shown at a time.

The detected color ID was then evaluated:

- If ID equals 1, the red LED channel was turned ON
- If ID equals 2, the green LED channel was turned ON
- If ID equals 3, the blue LED channel was turned ON

This logic ensures that the RGB LED visually represents the color detected by the HuskyLens camera in real time.

6. Timing Control

A delay of 1000 milliseconds was introduced after each detection cycle. This delay allows sufficient time for visual observation of the LED output, prevents rapid LED switching, and avoids excessive serial data transmission.

7. Overall Program Operation

The program continuously performs video interfacing through the HuskyLens AI Vision Camera, interprets detected color information, and provides immediate visual feedback using an RGB LED. This demonstrates effective integration of video input, serial communication, and output actuation in a microcontroller-based system.

Code Used:

```
#include <SoftwareSerial.h>
#include <DFRobot_HuskyLens.h>

SoftwareSerial mySerial(4, 5);  // RX, TX
HUSKYLENS huskylens;

// RGB LED pin definitions
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
```

```
  Serial.begin(9600);
  mySerial.begin(9600);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Turn off all colors initially
  digitalWrite(redPin, LOW);
  digitalWrite(greenPin, LOW);
  digitalWrite(bluePin, LOW);

  while (!huskylens.begin(mySerial)) {
    Serial.println("HuskyLens not connected!");
    delay(500);
  }

  Serial.println("HuskyLens Ready");
}

void loop() {
  if (!huskylens.request()) {
    Serial.println("Request failed");
    return;
  }

  if (huskylens.available()) {
    HUSKYLENSResult result = huskylens.read();

    Serial.print("ID: ");
    Serial.print(result.ID);
    Serial.print("  X: ");
    Serial.print(result.xCenter);
    Serial.print("  Y: ");
    Serial.println(result.yCenter);

    // Turn off all LEDs first
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, LOW);
```

```
  digitalWrite(bluePin, LOW);

  // Light up based on detected ID
  if (result.ID == 1) {
    digitalWrite(redPin, HIGH);      // RED ON
    Serial.println("→ Showing RED");
  }
  else if (result.ID == 2) {
    digitalWrite(greenPin, HIGH);    // GREEN ON
    Serial.println("→ Showing GREEN");
  }
  else if (result.ID == 3) {
    digitalWrite(bluePin, HIGH);     // BLUE ON
    Serial.println("→ Showing BLUE");
  }

  delay(1000);
 }
}
```

Control Algorithm

1. **Defining Pins**

RGB LED Outputs:

- Pin 9 is configured as the red LED output.
- Pin 10 is configured as the green LED output.
- Pin 11 is configured as the blue LED output.

HuskyLens Communication:

- SoftwareSerial communication is established using digital pin 4 as RX and digital pin 5 as TX for UART communication with the HuskyLens AI Vision Camera.

2. **Global Variables**

SoftwareSerial mySerial(4, 5)

- Handles serial communication between the Arduino Uno and the HuskyLens camera.

HUSKYLENS huskylens

- Used to initialize the camera, send detection requests, and read recognition results.

RGB LED pin constants

- Used to control individual color channels based on the detected color ID.

3. **System Initialization**
    - Serial communication is initialized at 9600 baud for monitoring and debugging.
    - SoftwareSerial communication is initialized at 9600 baud for HuskyLens data transmission.
    - RGB LED pins are configured as OUTPUT pins and set to LOW to ensure the LED is OFF during startup.
    - The system continuously attempts to establish communication with the HuskyLens camera until a successful connection is detected.
    - Once connected, a confirmation message is printed to indicate that the camera is ready for operation.

4. **Main Loop**

HuskyLens Data Request:

- The Arduino continuously sends a request to the HuskyLens to retrieve detection data.
- If the request fails, an error message is displayed and the loop iteration is terminated.

Object Detection and Data Reading:

- When detection data is available, the system reads the detected object's color ID and its X and Y center coordinates.
- The retrieved values are printed to the Serial Monitor for observation and verification.

5. **Decision Logic for RGB LED Output**

- All RGB LED channels are turned OFF at the start of each detection cycle to prevent color overlap.
- The detected color ID is evaluated using conditional statements:
    - ID = 1 → Red LED is turned ON
    - ID = 2 → Green LED is turned ON
    - ID = 3 → Blue LED is turned ON

This ensures that only one LED color is activated at any given time, corresponding to the detected object color.

6. **Output Feedback**

- The activated LED provides immediate visual confirmation of the detected color.
- A corresponding status message is printed on the Serial Monitor to indicate which color is currently displayed.

7. **Timing and Stability Control**

- A delay of 1000 milliseconds is applied at the end of each detection cycle.
- This delay ensures stable LED operation, allows sufficient observation time, and prevents excessive data requests to the HuskyLens.

8. **Overall Algorithm Behaviour**

The control algorithm continuously acquires video input from the HuskyLens AI Vision Camera, processes the detected color information, and translates it into a corresponding RGB LED output. This closed-loop process demonstrates effective video interfacing, decision-making logic, and output control within a microcontroller-based system.

DATA COLLECTION

Observation:

Detected Colour ID	Detected Colour	RGB LED Output	Result
1	Red	Red LED	Correct
2	Green	Green LED	Correct
3	Blue	Blue LED	Correct

DATA ANALYSIS

The HuskyLens color detection system's data demonstrates a clear and reliable correlation between the RGB LED output and the identified color ID. Accurate data interpretation and decision logic implementation were demonstrated by the proper activation of the corresponding LED channel for each identified color ID (1, 2, and 3).

During testing, there were no instances of improper LED activation or color overlap. At the start of every detection cycle, the system properly reset all RGB LED outputs, guaranteeing that only one LED color was illuminated at a time. This demonstrates that residual output from earlier detections is successfully prevented by the control mechanism.

By providing sufficient processing time and minimizing unnecessary data queries to the HuskyLens camera, the one-second delay included in each loop cycle helped to maintain stable system performance. This time control guaranteed consistent visual feedback and increased detection reliability.

Overall, the data analysis shows that, under typical working settings, the system was able to map detected color IDs to the associated RGB LED outputs with 100% accuracy. The outcomes show a dependable integration of serial communication, microcontroller output control, and AI-based visual detection.

RESULT

A color detection system utilizing HuskyLens was successfully put into operation and evaluated. An Arduino Uno and an RGB LED served as the output indicator. Communication between the Arduino and the HuskyLens AI Vision Camera was achieved using SoftwareSerial, and the system verified a successful connection during startup. In the testing phase, the HuskyLens successfully identified object colors and assigned the associated color identification numbers. The Arduino then received these color identification numbers and executed the programmed decision-making process. Upon detection of each color identification number, the corresponding RGB LED channel was enabled. Specifically, color identification number one triggered the red LED, color identification number two activated the green LED, and color identification number three activated the blue LED.

The LED reset logic successfully prevented colour overlap because only one LED colour was ever lit. Real-time system performance observation and verification were made possible by

the Serial Monitor's presentation of the detected colour ID and matching object coordinates. Overall, the findings show that the system consistently meets the project's stated functional requirements by converting visual colour recognition from the HuskyLens camera into precise and instantaneous RGB LED output.

DISCUSSION

As each detected colour ID reliably triggered the appropriate RGB LED output, the experimental findings show that the HuskyLens-based colour detection system operates efficiently. This verifies that the developed control logic operates as expected and shows dependable serial connectivity between the Arduino Uno and the HuskyLens AI Vision Camera.

Colour overlap was effectively avoided by resetting all RGB LED channels at the beginning of each detection cycle, guaranteeing precise and understandable visual feedback. By restricting excessive data requests to the HuskyLens and providing sufficient observation time for the LED output, the addition of a one-second delay enhanced system stability. However, in applications that need quicker detection, this delay lowers system responsiveness.

Environmental factors including ambient illumination, object distance, and colour similarity may have an impact on system performance. Changes in these variables may affect the precision of the detection. Overall, the system achieves its goals of combining microcontroller control with AI-based vision sensing, and it has prospects for future improvements in detection robustness and response speed.

CONCLUSION

In conclusion, an Arduino Uno and an RGB LED were used to successfully build and construct the HuskyLens-based colour detecting system. Accurate colour ID identification and interpretation were made possible by the system's dependable microcontroller-HuskyLens AI Vision Camera connection.

The findings of the trial verified that every colour ID that was recognised was accurately mapped to its matching RGB LED output, offering instantaneous and clear visual feedback. The control algorithm that was put into place successfully avoided colour overlap and guaranteed steady system performance. The system achieved all of the project's goals, despite the response time being constrained by the enforced delay and external elements like lighting.

Overall, the project demonstrates the effective integration of AI vision sensing with microcontroller-based control and provides a strong foundation for further improvements and more advanced applications.

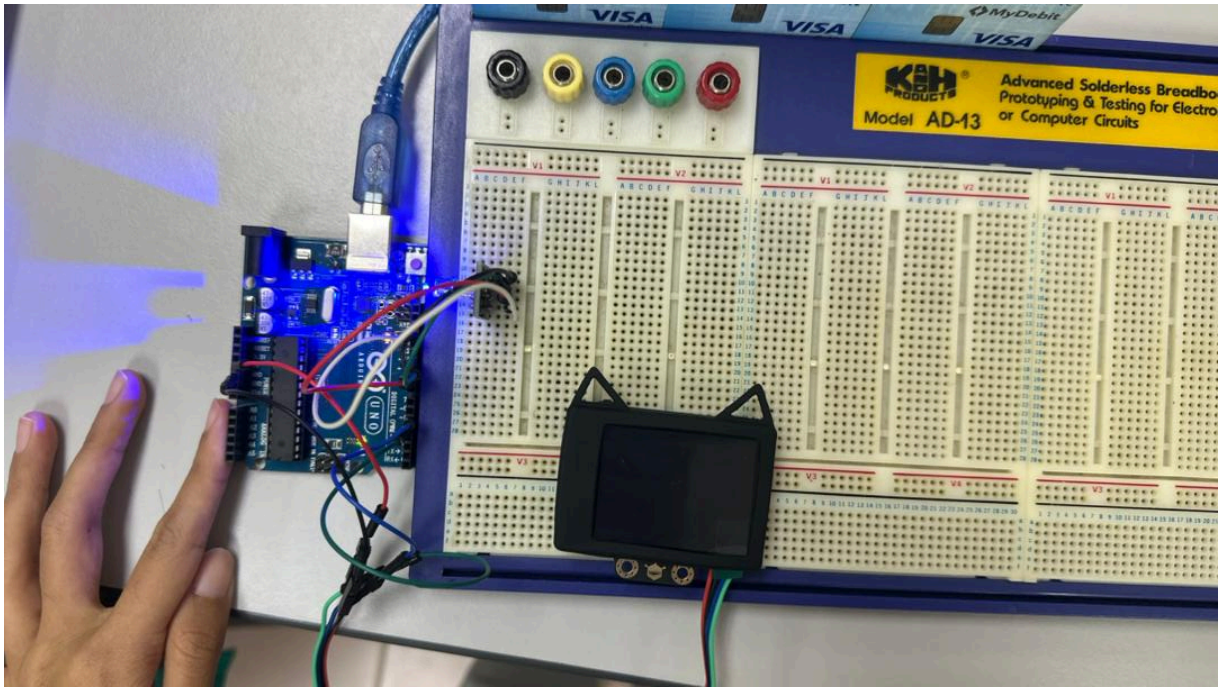
RECOMMENDATIONS

Based on the experimental results, several improvements can be recommended to enhance the performance and applicability of the HuskyLens-based colour detection system. Consistent and controlled lighting conditions should be implemented, such as using dedicated light sources or an enclosure, to minimize the effects of shadows and ambient light variations on colour recognition accuracy. Expanding the number of trained colours and applying verification or filtering techniques before LED activation could further improve detection reliability, especially when dealing with similar colour tones. For better usability, the integration of an external display such as an LCD or OLED module is recommended to provide real-time visual feedback without relying solely on the Serial Monitor. Additionally, incorporating other output devices or system extensions, such as motors or communication modules, would allow the system to be applied in more advanced mechatronic applications, including automated sorting and intelligent robotic systems.

REFERENCES

DFRobot. (n.d.). *Gravity: HUSKYLENS AI machine vision sensor*. DFRobot Wiki. https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336

APPENDICES



Circuit for Experiment 9

ACKNOWLEDGEMENTS

Hereby, we acknowledge the guidance provided by Dr Zulkifli with his impeccable knowledge in this field. We also thank Br. Harith with his assistance in correcting our work thus leading to a successful experiment. Not to forget our fellow colleagues from other groups that contributed in providing ideas that helped in achieving the same output together from this experiment.

STUDENTS DECLARATION

Certificate of Originality and Authenticity

We hereby certify that we are responsible for the work presented in this report. The content is our original work, except where proper references and acknowledgements are made. We confirm that no part of this report has been completed by anyone not listed as a contributor. We also certify that this report is the result of group collaboration and not the effort of a single individual. The level of contribution by each member is stated in this certificate. Furthermore, we have read and understood the entire report, and we agree that no further revisions are required. We collectively approve this final report for submission and confirm that it has been reviewed and verified by all group members.

Signature:  Read [/]

Name: HARIZ IRFAN BIN MOHD ROZHAN Understand [/]

Matric Number: 2318583 Agree [/]

Signature:  Read [/]

Name: ABDULLAH HASAN BIN SIDEK Understand [/]

Matric Number: 2318817 Agree [/]

Signature:  Read [/]

Name: TAN YONG JIA Understand [/]

Matric Number: 2319155 Agree [/]

Signature:  Read [/]

Name: NUR QISTINA BINTI MOHD FAIZAL Understand [/]

Matric Number: 2319512 Agree [/]